Apertura: lunes, 9 de noviembre de 2026, 00:00
Cierre: lunes, 16 de noviembre de 2026, 14:00

UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR	VICERRECTORADO DOCENTE	**Código:** GUIA-PRL-001
	CONSEJO ACADÉMICO	**Aprobación:** 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR	PRÁCTICA DE LABORATORIO

CARRERA: Computación	ASIGNATURA: Computación Paralela
NRO. PRÁCTICA: 02	TÍTULO PRÁCTICA: Convolución de imágenes de alta resolución mediante PyCUDA

OBJETIVOS

- Adaptar los filtros de convolución de la práctica 01 a Python y ejecutar sus kernels en la GPU mediante PyCUDA.
- Verificar la equivalencia de los resultados y comparar el rendimiento de C/C++, CUDA C, Python secuencial y PyCUDA.
- Comunicar resultados experimentales reproducibles.

?

INSTRUCCIONES

Modalidad: trabajo en parejas. **Calificación:** 5 puntos.

Esta práctica continúa el trabajo desarrollado en la práctica 01. Cada pareja migrará a Python la preparación de imágenes, la gestión de datos, las mediciones y el almacenamiento de resultados, e integrará PyCUDA para ejecutar en la GPU los kernels de convolución previamente implementados.

Se conservarán las imágenes, filtros y condiciones de procesamiento de la práctica anterior. Se incorporará una implementación secuencial en Python y se compararán cuatro versiones: **C/C++ secuencial, CUDA C, Python secuencial y PyCUDA**.

En PyCUDA, el programa anfitrión se escribe en Python y el kernel se mantiene en CUDA C. El propósito es comprender cómo se prepara y transfiere la información, cómo se configura el lanzamiento y cómo se recuperan y verifican los resultados desde Python.

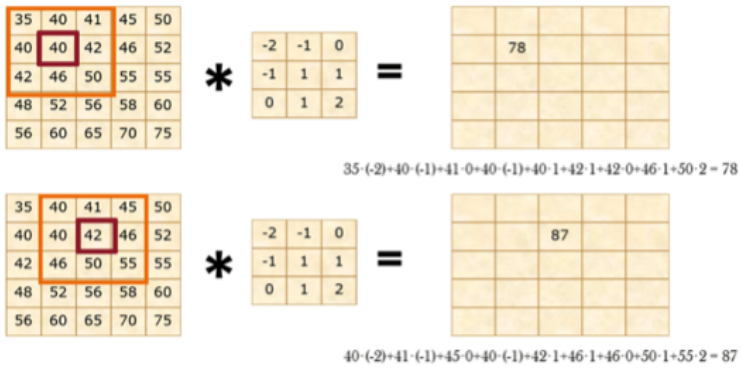


Figura 1. Aplicación de una máscara sobre una vecindad. Imagen conservada de la guía original.

El esquema ilustra la multiplicación y acumulación sobre una ventana. Formalmente, la convolución invierte la máscara respecto de la correlación; las máscaras simétricas propuestas en esta práctica producen el mismo resultado en ambos casos.

El suavizado gaussiano atenúa variaciones locales. La diferencia de gaussianas resta dos versiones suavizadas con distinta escala y permite resaltar cambios de intensidad. Las siguientes imágenes son referenciales; este filtro no forma parte de los tres filtros exigidos.

Imagen original	**Diferencia de gaussianas**
	

Figuras 2 y 3. Ejemplo conservado de la guía de práctica original proporcionada por el docente.

Requisitos de implementación

1. **Continuidad:** reutilizar los filtros de media, nitidez y laplaciano de la práctica 01, sus coeficientes y las imágenes de 2000×2000, 4000×4000 y 6000×6000. Conservar la versión anterior para la comparación e identificar su commit.
2. **Programa en Python:** implementar la carga, conversión a escala de grises, preparación de máscaras y arreglos, selección del filtro, ejecución, validación, exportación de tiempos y guardado de resultados.
3. **Convolución secuencial:** implementar en Python la misma operación mediante recorridos explícitos de píxeles y coeficientes. NumPy puede utilizarse para almacenar datos y calcular errores; no sustituir la convolución evaluada por funciones de filtrado ya implementadas, JIT, multiprocesamiento ni llamadas a la versión en C.
4. **Convolución con PyCUDA:** adaptar el kernel de la práctica 01, compilarlo y obtener su función desde Python. Gestionar memoria y transferencias, pasar argumentos con tipos compatibles y configurar explícitamente bloques e hilos. Utilizar `SourceModule` o una carga equivalente documentada.
5. **Distribución:** mantener bloques e hilos en la dimensión X y un píxel de salida por hilo. Leer los vecinos desde memoria global; explicar la conversión del índice global a fila y columna, controlar límites y separar los arreglos de entrada y salida.

6. **Equivalencia:** emplear la misma precisión de datos y acumulación, extensión con ceros y tratamiento del resultado en las cuatro versiones. Verificar la disposición contigua de los arreglos transferidos y documentar los tipos usados en NumPy y en el kernel.
7. **Implementación propia:** la convolución GPU evaluada debe ejecutarse con el kernel propio mediante PyCUDA. Las bibliotecas de imágenes se pueden usar para lectura, conversión y escritura.

Filtros y máscaras

Filtro	Configuración
Media	Máscara k×k con coeficientes 1/(k×k). Evaluar k = 3, 9 y 21.
Nitidez	3×3: [0, −1, 0; −1, 5, −1; 0, −1, 0].
Laplaciano	3×3: [0, 1, 0; 1, −4, 1; 0, 1, 0].

Se mantienen las máscaras de la práctica 01 actualizada. La variación del tamaño de máscara se estudia con el filtro de media; nitidez y laplaciano se conservan en 3×3.

Preparación y validación

Usar exactamente los archivos de entrada de la práctica 01 y registrar CPU, núcleos, RAM, GPU, memoria de GPU, sistema operativo y versiones de Python, NumPy, PyCUDA, controlador y CUDA Toolkit. Documentar cómo se prepara el entorno.

- **Prueba pequeña:** comprobar manualmente una matriz y verificar las cuatro implementaciones.
- **Prueba de límites:** procesar una imagen cuyo número de píxeles no sea múltiplo de los hilos por bloque; comprobar bordes y cobertura de todos los píxeles.
- **Comparación numérica:** usar C/C++ secuencial validado como referencia y comparar Python secuencial, CUDA C y PyCUDA. Registrar error absoluto máximo y medio antes del recorte al rango de visualización. Definir una tolerancia y justificarla según la precisión utilizada.
- **Evidencia visual:** incluir la imagen original y resultados de los tres filtros. Mostrar un recorte de la misma región cuando el efecto no sea perceptible en la imagen reducida.

Experimentos obligatorios

Experimento	Configuración	Versiones
Filtros	Media, nitidez y laplaciano de 3×3; imagen de 4000×4000.	Las cuatro versiones.
Resolución	Media de 9×9; imágenes de 2000×2000, 4000×4000 y 6000×6000.	Las cuatro versiones.
Máscara	Media de 3×3, 9×9 y 21×21; imagen de 4000×4000.	Las cuatro versiones.
Hilos por bloque	Media de 9×9; 4000×4000; bloques de 128, 256 y 512 hilos, si el equipo lo permite.	CUDA C y PyCUDA.

Modificar un factor a la vez. Mantener fija la configuración de hilos fuera del experimento específico y ajustar la cantidad de bloques a la imagen. Las configuraciones repetidas pueden compartir mediciones cuando las condiciones sean idénticas.

Los tiempos anteriores de C/C++ y CUDA C solo podrán reutilizarse si corresponden al mismo equipo, datos, código y alcance de medición. Si cambian las condiciones, repetir esas ejecuciones y documentarlo.

Viabilidad de Python secuencial: ejecutar primero una prueba pequeña para estimar el tiempo. Si una configuración grande no puede completarse en el tiempo de laboratorio, registrar el tiempo transcurrido, la configuración y el motivo de interrupción. Marcarla como «no completada»; no presentar estimaciones como tiempos medidos. Completar una comparación de las cuatro versiones en una resolución menor común y mantener las pruebas de alta resolución en PyCUDA. Una limitación documentada con evidencia y esta comparación alternativa no se penaliza como prueba omitida.

Medición del rendimiento

Realizar calentamiento previo y al menos **cinco repeticiones completas** por configuración viable. Reportar mediana y variabilidad. Aplicar el procedimiento de limitación documentada cuando corresponda.

- **CPU:** medir únicamente la convolución secuencial. En Python utilizar un reloj de alta resolución, como `time.perf_counter()`.
- **GPU:** registrar por separado transferencia CPU→GPU, kernel, transferencia GPU→CPU y tiempo conjunto de transferencias y ejecución. Medir el kernel con eventos CUDA desde PyCUDA y sincronizar antes de consultar el tiempo.
- **Preparación:** excluir lectura/escritura de imágenes y visualización. Registrar por separado inicialización del contexto y compilación/carga del kernel; no repetirlas dentro de cada medición del kernel. Declarar si la reserva de memoria forma parte del tiempo conjunto.
- **Registros:** entregar CSV con versión, filtro, resolución, máscara, hilos/bloques, repetición, tiempos, unidades y estado de la ejecución.

Calcular la aceleración CUDA C respecto a C/C++ secuencial y la aceleración PyCUDA respecto a Python secuencial, tanto para el kernel como incluyendo transferencias. Comparar además los tiempos de CUDA C y PyCUDA bajo condiciones equivalentes. No calcular aceleraciones con ejecuciones incompletas.

Análisis de resultados

1. ¿Qué partes del programa se trasladaron a Python y qué partes del kernel se reutilizaron?
2. ¿Las cuatro versiones producen resultados equivalentes? Explicar las diferencias numéricas encontradas.
3. ¿Cómo influyen la resolución y la máscara en los tiempos de Python secuencial y PyCUDA?
4. ¿Qué diferencias existen entre CUDA C y PyCUDA al medir únicamente el kernel y al incluir transferencias y gestión desde el host?
5. ¿Qué configuración de hilos por bloque obtuvo mejores resultados y con qué variabilidad?
6. ¿Qué limitaciones del equipo o del procedimiento afectan las conclusiones?

PyCUDA ejecuta kernels CUDA en la misma GPU; no se debe asumir que será más rápido que CUDA C. La comparación de las versiones secuenciales también refleja diferencias entre lenguajes y entornos de ejecución. Fundamentar las conclusiones en los registros y distinguir los resultados medidos de las hipótesis.

Entrega y repositorio

1. **Informe PDF:** contexto y relación con la práctica 01, explicación de la migración, entorno, flujo de ejecución, validación, imágenes, tablas, gráficas comparativas, análisis, conclusiones, contribuciones y firmas de ambos integrantes.
2. **GitHub:** programa Python secuencial, programa PyCUDA, kernels propios, dependencias con versiones, README con comandos de instalación y ejecución, datos o instrucciones para obtenerlos, pruebas y CSV. Conservar C/C++ y CUDA C de la práctica 01 en una carpeta identificada o enlazar su commit.
3. **Trazabilidad:** incluir el enlace accesible al repositorio y el commit exacto de la práctica 02. Distinguir resultados nuevos y reutilizados. El código entregado debe corresponder a las mediciones del informe.
4. **AVAC:** cargar el informe y las diapositivas de la presentación; el informe debe contener los enlaces al código y los datos.

Presentación y defensa

10 minutos de exposición y 5 minutos de preguntas. Ambos integrantes deberán participar y comprender la solución completa. Explicar la migración, la gestión de datos desde Python, el lanzamiento del kernel, la validación y las comparaciones. El docente podrá solicitar una demostración o la explicación de fragmentos del código.

RÚBRICA DE EVALUACIÓN — 5 PUNTOS

Criterio	Máximo	Logro completo	Logro parcial	Logro insuficiente
Migración e implementación	1,0	1,0: Python secuencial y PyCUDA ejecutan los tres filtros; kernel propio, tipos, transferencias, bloques, límites y bordes correctos.	0,5: Implementación funcional parcialmente, con omisiones o errores localizados.	0–0,25: Migración incompleta o convolución PyCUDA no verificable.
Validación de resultados	0,75	0,75: Prueba manual y de límites; comparación de las cuatro versiones; errores numéricos y tolerancia justificada.	0,375: Verificación útil con pruebas o explicaciones incompletas.	0–0,125: Solo evidencia visual o ausencia de validación numérica.
Experimentos y medición	0,75	0,75: Configuraciones y CSV completos, calentamiento, repeticiones, variabilidad y tiempos separados; límites documentados según la guía.	0,375: Mediciones útiles con omisiones metodológicas o de registros.	0–0,125: Tiempos sin respaldo o alcance de medición indefinido.
Análisis comparativo	0,75	0,75: Interpreta las cuatro versiones, aceleraciones, tamaños y transferencias; distingue kernel y gestión desde Python y reconoce limitaciones.	0,375: Describe tendencias, pero explica parcialmente sus causas.	0–0,125: Solo muestra gráficas o afirma ventajas sin evidencia.
Informe y reproducibilidad	0,75	0,75: Informe coherente, commit identificado, dependencias, comandos, datos y pruebas suficientes; contribuciones documentadas.	0,375: Revisable con documentación o trazabilidad incompleta.	0–0,125: No permite verificar una parte sustancial del trabajo.
Presentación de resultados	0,5	0,5: Ambos participan; migración y hallazgos claros; gráficas legibles y tiempo respetado.	0,25: Comunica resultados con dificultades de claridad, organización o participación.	0–0,125: Presentación incompleta o sin resultados claros.

Criterio	Máximo	Logro completo	Logro parcial	Logro insuficiente
Defensa y dominio	0,5	0,5: Ambos explican el flujo Python–GPU, las decisiones y los resultados y responden con fundamento.	0,25: Comprensión parcial o dificultad para relacionar código y resultados.	0–0,125: No explican elementos esenciales de su solución.
TOTAL	5,0	Informe y código: 4 puntos.	Presentación y defensa: 1 punto.	

La calificación es global para la pareja. La ausencia de evidencia corresponde a 0 puntos en el criterio. Dentro de los rangos se valorará la evidencia disponible. Se evalúan corrección, análisis y reproducibilidad, sin exigir una aceleración determinada. La presentación y defensa se califican durante la exposición; las diapositivas por sí solas no sustituyen esa evidencia.

Material de apoyo

Práctica 01 y guía referencial de práctica 02 proporcionadas por el docente. Consulta técnica: Tutorial oficial de PyCUDA y interfaz de dispositivo y eventos.

DESARROLLO Y CIERRE

ACTIVIDADES POR DESARROLLAR

1. Revisar la práctica 01 y preparar el entorno Python/PyCUDA.
2. Migrar el programa anfitrión e implementar Python secuencial.
3. Integrar y ejecutar los kernels con PyCUDA.
4. Validar las cuatro versiones y ejecutar los experimentos.
5. Consolidar CSV, gráficas, informe y repositorio.
6. Presentar y defender los resultados en pareja.

RESULTADO(S) OBTENIDO(S)

- Integra kernels CUDA en una aplicación Python mediante PyCUDA.
- Verifica la equivalencia y compara tiempos entre implementaciones.
- Comunica resultados reproducibles y sus limitaciones.

CONCLUSIONES

Explicar qué cambia al gestionar la ejecución desde Python y qué se conserva en la GPU. Sustentar los hallazgos con la validación y los registros experimentales.

RECOMENDACIONES

Comenzar con entradas pequeñas; comprobar tipos, límites y sincronización; reservar tiempo para Python secuencial. Conservar los resultados y el código de ambas prácticas y revisar el trabajo entre los dos integrantes.

Docente: Ing. Gabriel A. León Paredes, PhD.
Firma: G.A.L.P
Resolución CS N° 076-04-2016-04-20

Agregar entrega

Estado de la entrega

Grupo	Grupo A
Estado de la entrega	No se ha enviado nada en esta tarea
Estado de la calificación	Sin calificar
Tiempo restante	6 días 1 hora restante